



UNIVERSITI
MALAYA

WIA1002 DATA STRUCTURE

Smart Library Project

TECHNICAL REPORT

SEM II 2025/2026

DR. MOHAMED N. M. LUBANI

OCCURRENCE 10, TEAM 2:

CHEN BINGYAN	24078302
BEBY AZZURA	24201647
YUDONG CHEN	24074901
HILAL MUMTAZ SALEH HARAHAHAP	24200675
NAZEEF NUWAISIR KHAN	24085530

Github Link: https://github.com/24078302Liz/DataStructure_Tutorial_Lab_sem2.git

Table of Contents

Table of Contents	2
1. Introduction & Executive Summary	3
1.1 Team Distribution & Responsibility Matrix	3
2. Abstract Data Type (ADT) Design & Information Hiding	5
3. Catalogue Architecture: Binary Search Tree (BST)	6
3.1 Node Object Design & Field Layout	6
3.2 Recursive Insertion Optimization	6
3.3 Logarithmic Search Path Operations	7
3.4 Multi-Scenario Node Deletion (The Borrowing Trigger)	7
4. Transaction Registry: Stack Design	8
4.1 Encapsulation of the Legacy Collection Layer	8
4.2 Null-Safety Boundaries & Input Defenses	8
4.3 Non-Destructive Reverse-Chronological Rendering Logic	8
5. System Integration & Console UI Layer	9
5.1 Atomic Borrowing Lifecycle Flow	9
5.2 Robust Type-Parsing & Error Recovery Scanners	10
6. Algorithmic Complexity & Performance Analysis	10
6.1 Data Structure Operation Complexity Matrix	10
6.2 Addressing Potential Structural Bottlenecks	11
7. Verification Matrix & Quality Assurance Summary	12
7.1 Comprehensive Quality Assurance Testing Ledger	12
8. Conclusion	13

1. Introduction & Executive Summary

The Smart Library Management System is a terminal-based tool built to handle the everyday administrative work of a university library. Rather than reaching for off-the-shelf solutions, the project was designed from the ground up using object-oriented principles, partly as a practical exercise, and partly to answer a genuine question: what happens when you pit different data access strategies against each other inside the same system?

One of the core tensions in any library software is that not all data behaves the same way. The catalogue needs to answer the question *"Does this book exist?"* almost instantly, no matter how large the collection grows. Transaction records, on the other hand, need to preserve a reliable, chronological trail of who borrowed what and when. These two requirements pull in opposite directions, and bridging them is where things get interesting.

To handle this, the system uses a dual-engine approach where each layer of data is managed by a structure suited to its specific job.

- **The Catalogue Inventory:** The book catalogue is managed entirely through a custom Binary Search Tree. This means that whether you're adding a new title or searching for one by ISBN, the system doesn't have to scan through everything it knows. It narrows down to the right answer in logarithmic time, so even as the collection grows deeper, lookups stay fast.
- **The Borrowing Registry Ledger:** The Borrowing Registry Ledger is governed by a stack, meaning the most recent activity always surfaces first. This mirrors how people naturally think about their borrowing history, where what just happened matters more than something from three weeks ago. To keep the modules working together without them becoming too tangled up in each other's internals, the team introduced an Abstract Data Type (ADT) interface layer as a kind of shared contract between components. The rest of this report walks through how the system was built, how each class fits together, how the algorithms perform, and how the whole thing held up under testing.

1.1 Team Distribution & Responsibility Matrix

Building the Smart Library was a team effort, with each member taking ownership of a specific part of the system so work could move forward in parallel without stepping on each other's toes:

Engineer	Role Assignment	Core Architectural Responsibility
Nazeef	Catalogue Architect	Engineered the node topology (Book class) and structural BST injection layer; developed the recursive leaf and branch node balancing insertion mechanics.
BingYan	Borrowing History	Constructed the BorrowStack transactional layer; enforced strict null-safety boundaries and implemented non-destructive reverse-index stack traversal.
YuDong	Record Finder	Designed and optimized the recursive lookup algorithms within the hierarchical catalogue tree, establishing key-matching conditional branches.
Hilal	ADT Designer	Authored the structural contract (LibraryADT); governed architectural compliance, encapsulation audits, and verified global information hiding properties.
Rara	Integration & UI	Wrote the main loop console harness, implemented safe token-based input scanners, built the type-parsing error-recovery blocks, and fused the BST-to-Stack borrowing transaction workflow.

2. Abstract Data Type (ADT) Design & Information Hiding

Software robustness depends on the restriction of access vectors. Following the software engineering principle of Information Hiding, the internal states of data structures must remain unexposed to client-side routines. The application guarantees this security boundary via `LibraryADT.java`, an interface defining the operational capabilities of the library backend without leaking its architectural details.

The LibraryADT Interface Specification:

```
public interface LibraryADT {
    void addBook(int isbn, String title, String author);
    Book SearchBook(int isbn);
    void borrowBook(int isbn);
    void viewLatestHistory();
}
```

By mandating that the orchestration class (**SmartLibrary.java**) implements this interface, the client UI layer (**Main** and the menu routines) is structurally restricted. It can only interact with the data through these four public standard endpoints. To verify complete security isolation, the following design constraints were systematically enforced across the codebase:

1. **Encapsulated Tree Pointers:** The root node pointer in **BookBST.java** is strictly marked as *private Book root;*. Outside classes cannot traverse or reassign the root node, preventing corruption of the tree topology.
2. **Protected Stack Arrays:** The underlying storage collection inside **BorrowStack.java** is defined as *private Stack<Book> stack;*. This thwarts clear vectors of malicious or accidental index wiping, pop overruns, or manual insertion of out-of-order logs.
3. **Complete State Isolation:** The main instance references inside **SmartLibrary**—namely *catalogue* and *history*—are declared private. The UI layer can never directly invoke tree or stack methods; it relies entirely on the unified business logic of the overridden interface routines.

3. Catalogue Architecture: Binary Search Tree (BST)

The core collection inventory is managed by a Binary Search Tree specifically optimized for quick lookups using a unique identifier. Rather than separating the data payload from the structural linkage, the system uses an elegant architecture where `Book.java` acts as both the object-oriented business entity and the tree node.

3.1 Node Object Design & Field Layout

Each instance of the `Book` class maintains encapsulated parameters representing its bibliographic values, alongside a set of references that represent the left and right paths within the tree structure:

- **private int isbn:** The unique International Standard Book Number. This primitive integer serves as the exclusive sorting key used across all tree balancing comparisons.
- **private String title / private String author:** The bibliographic metadata payload stored directly at the node.
- **private Book left / private Book right:** Self-referential node pointers targeting subtrees containing books with smaller or larger keys, respectively.

3.2 Recursive Insertion Optimization

The arrangement of the tree is maintained dynamically during record insertion via the *recursiveInsert(Book currentNode, Book newBook)* mechanism. This method employs a divide-and-conquer branching paradigm:

- **Duplicate Verification Base Case:** Before any subtree injection occurs, the system matches `newIsbn == currentIsbn`. Upon intercepting a match, it rejects the insert command with an error message, preserving the absolute key integrity of the index.
- **Left Subtree Traversal:** If the incoming ISBN is smaller than the current node's value, the system inspects `currentNode.hasLeft()`. If true, it recurses downward; if false, it links the new node into that available space.
- **Right Subtree Traversal:** If the incoming ISBN exceeds the current node's value, it mirrors the left side logic on the right branch.

3.3 Logarithmic Search Path Operations

The main benefit of this catalogue design is its quick search capability, developed within *recursiveSearch(Book currentNode, int isbn)*. By comparing the search key against the current node's ISBN, the algorithm completely rules out half of the remaining search space at every step. This leads to an average time complexity of $O(\log n)$. The function relies on two clear base cases and two recursive directions:

1. *currentNode == null* → Reached a leaf dead-end; returns null immediately, signifying the book does not exist.
2. *currentNode.getIsbn() == isbn* → Key target found; returns the book reference to the caller.
3. *isbn < currentNode.getIsbn()* → Recurses down the left branch.
4. *isbn > currentNode.getIsbn()* → Recurses down the right branch.

3.4 Multi-Scenario Node Deletion (The Borrowing Trigger)

When a book is borrowed, it must be removed from the catalogue tree using the *recursiveDelete(Book currentNode, int isbn)* method. Deleting a node from a BST is complicated because the remaining elements must be re-linked to preserve the sorting properties. The system handles all three classic structural deletion scenarios:

- **Scenario 1: Removing a Leaf Node (No Children).** If the book node has neither left nor right links, the function removes it simply by returning *null* to its parent pointer link.
- **Scenario 2: Removing a Node with a Single Child.** If the node lacks a left branch (*!currentNode.hasLeft()*), it returns its right child reference. If it lacks a right branch (*!currentNode.hasRight()*), it returns its left child. This effectively promotes the single child up a level to replace the deleted node.
- **Scenario 3: Removing a Node with Two Children (In-Order Successor).** When a node has two branches, removing it completely requires finding its **In-Order Successor**—the smallest value in its right subtree. The system invokes *getMinValueNode(currentNode.getRight())*, which tracks down the leftmost leaf of the right subtree. The values are then cloned, overwriting the data fields, and the duplicate successor node is recursively removed from the right subtree branch.

4. Transaction Registry: Stack Design

The `BorrowStack.java` class tracks user borrowing operations. Unlike a catalogue, which requires structured lookups, transaction ledgers are inherently chronological. Users care most about reviewing their recent activity. A Last-In-First-Out (LIFO) Stack perfectly mirrors this real-world workflow.

4.1 Encapsulation of the Legacy Collection Layer

To prevent unauthorized structural modifications, the system wraps Java's built-in `java.util.Stack` inside a controlled custom object wrapper. This ensures that arbitrary index shifts or mid-collection element deletions are completely blocked, exposing only the safe operations required by the library architecture.

4.2 Null-Safety Boundaries & Input Defenses

One thing that can quietly break a system is letting a null reference slip through into a deeper layer. To guard against this, the stack checks what it receives before doing anything with it:

```
public void push(Book book) {  
    if (book != null) {  
        stack.push(book);  
    } else {  
        System.out.println("Error: Cannot add a null book to history.");  
    }  
}
```

This structure forms an absolute guard rail. If an upstream search failure occurs or a corrupted entity is passed, the stack rejects the null pointer. This avoids a runtime *NullPointerException* during subsequent output rendering operations.

4.3 Non-Destructive Reverse-Chronological Rendering Logic

The transaction history needs to show the most recently borrowed book first, but iterating over a Java Stack the normal way prints things from oldest to newest. Calling `pop()` in a loop would fix

the order but wipe the history in the process. To get around both problems, the `show()` method uses a reverse index loop that reads the stack from top to bottom without touching its contents:

```
for (int i = stack.size() - 1; i >= 0; i--) {  
    Book b = stack.get(i);  
    System.out.println("[ISBN: " + b.getIsbn() + "] " + b.getTitle());  
}
```

This way, the most recent transaction always appears at the top of the output, and the stack itself stays completely untouched for anything that comes after.

5. System Integration & Console UI Layer

`SmartLibrary.java` is where everything comes together. It connects the catalogue and the borrowing history into one cohesive system, translating whatever the user does in the terminal into the right actions across the underlying data structures.

5.1 Atomic Borrowing Lifecycle Flow

The core feature of the system is the checkout transaction loop, which is encapsulated within *`borrowBook(int isbn)`*. This method links the individual modules to create an atomic transaction flow:

1. **Query Despatch:** Invokes the search algorithm on the BST catalogue to locate the target book record.
2. **History Preservation:** If found, the reference is pushed directly to the history stack ledger.
3. **Catalogue Excision:** The book record is immediately removed from the BST via recursive deletion, unlinking its nodes.
4. **Fault Handling:** If the book does not exist or is already checked out, it displays a clear notice to the user, skipping the deletion step.

5.2 Robust Type-Parsing & Error Recovery Scanners

One of the trickier parts of building a console program is handling user input that doesn't come in the expected format. Relying on something like `Scanner.nextInt()` is risky because typing the wrong thing can either corrupt the input buffer or crash the program entirely. To get around this, Han rewrote the input handling to read everything as a string first and then convert it:

```
String menuInput = sc.nextLine();
int choice;
try {
    choice = Integer.parseInt(menuInput.trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid option. Please enter a number.");
    continue;
}
```

This same approach is used everywhere the program expects a number, both in the main menu and when entering ISBNs. So if someone types something like "abc" by mistake, the program catches it, prints a short warning, and loops back to the menu cleanly without losing anything.

6. Algorithmic Complexity & Performance Analysis

One of the more important things to look at in any system like this is how it holds up as the data grows. The short answer is that it depends entirely on which structure is doing the work, so it's worth breaking down each one.

6.1 Data Structure Operation Complexity Matrix

System Operation	Underlying Structure	Asymptotic Complexity	Algorithmic Rationale & Boundaries
Add Book	BST Dynamic Insertion	Avg: $O(\log n)$ Worst: $O(n)$	On balanced trees, each step eliminates half the remaining paths. If keys are added in strictly sorted order,

			the tree becomes a linear linked list, increasing cost to linear time.
Search Book	BST Key Comparison	Avg: $O(\log n)$ Worst: $O(n)$	Logarithmic depth ensures fast lookups by comparing keys. Performance degrades to linear if the structure becomes heavily skewed.
Borrow Book	BST Search + Delete + Stack Push	Avg: $O(\log n)$	Combines a search lookup, tree re-linking, and a constant time stack push. The logarithmic search steps dominate the overall operation cost.
View History	Sequential Stack Array Traversal	Strictly $O(k)$	Requires rendering every borrowed entry exactly once, where k represents the current size of the checkout ledger history.

6.2 Addressing Potential Structural Bottlenecks

A Binary Search Tree works well when data comes in randomly, but it has one known weak spot: it's sensitive to the order books are added. If an administrator loads in a batch of ISBNs that are already sorted, say 101, 102, 103, 104, the tree never branches. It just grows in a straight line to the right, which effectively turns it into a linked list and drags search performance from $O(\log n)$ down to $O(n)$.

The straightforward fix for this in future versions would be switching to a self-balancing tree like an AVL Tree or a Red-Black Tree. These structures automatically reorganise themselves during insertions and deletions to stay balanced, which keeps lookups fast even when the incoming data isn't in an ideal order.

7. Verification Matrix & Quality Assurance Summary

To make sure everything worked as intended, the team ran the system through a full round of testing covering every feature, every input boundary, and every edge case we could think of. The goal was to be confident that nothing would break under normal use or unexpected inputs.

7.1 Comprehensive Quality Assurance Testing Ledger

Target Case	Input Payload	Expected Architectural Response	Assessment Status
Invalid Menu Option	9 or text choice	Intercepts input token, prints a numeric range warning, and safely loops back without crashing.	PASS (✓)
Non-Integer ISBN	abc	Catches NumberFormatException, prints a validation notice, and returns to main prompt.	PASS (✓)
Empty Metadata	Blank title/author	Trims input string, identifies length as zero, and rejects record addition.	PASS (✓)
Valid Book Addition	101, Java Basics, Liang	Constructs a new Book node and successfully injects it into the matching BST path.	PASS (✓)
Duplicate ISBN Check	101 again	Identifies existing matching key and blocks duplicate insertion to protect data integrity.	PASS (✓)
Search Existing Book	101	Traverses branches, finds key match, and renders complete bibliographic fields.	PASS (✓)

Search Missing Book	999	Reaches a null node pointer reference and returns a clean 'Book not found' message.	PASS (✓)
Borrow Existing Book	101	Pushes record pointer to history stack and runs multi-scenario node deletion on the BST catalogue.	PASS (✓)
Borrow Missing Book	999	Fails initial search query, displays an error message, and skips stack mutation steps.	PASS (✓)
View Empty History	Query prior to checkout	Identifies stack size as zero and prints a clean 'History is empty' summary notice.	PASS (✓)
View LIFO History	Checkout order: 101, 102, 103	Iterates backwards from index size-1, rendering item 103 first, then 102, then 101.	PASS (✓)

8. Conclusion

The Smart Library Management System shows what's possible when you match the right data structure to the right problem. By pairing Binary Search Trees with a Stack, the system handles two very different demands cleanly: catalogue lookups stay fast no matter how many books are added, and borrowing history always surfaces in the order that makes sense to the user.

Beyond the data structures themselves, the decision to enforce an ADT interface and build in proper input validation meant the system stayed predictable and didn't fall apart at the edges. Ultimately, this project reinforced a simple but easy-to-overlook idea: getting the data structure right from the start is what makes everything else easier to build on top of.